TRƯỜNG ĐẠI HỌC SƯ PHẠM, ĐẠI HỌC HUẾ
KHOA TOÁN HỌC

ĐỀ TÀI:

ỨNG DỤNG SYMPY TRONG GIẢI HỆ PHƯƠNG TRÌNH

Thành viên: Hoàng Thúy An
Lê Vũ Kim Hồng
Lê Phan Quỳnh Trang
Nguyễn Vy Thảo
Nhóm: 9

Huế, 9/2026

Mục lục

LỜI NÓI ĐẦU **2**

1 Chương 1. GIỚI THIỆU CHUNG VỀ SYMPY VÀ GIẢI HỆ PHƯƠNG TRÌNH **3**

1.1 Giới thiệu về SymPy . . . 3

1.1.1 SymPy là gì? . . . 3

1.1.2 Đặc điểm của SymPy . . . 3

1.1.3 Ưu điểm khi sử dụng SymPy trong tính toán toán học . . . 3

1.1.4 Một số lĩnh vực có thể sử dụng SymPy . . . 4

1.2 Giới thiệu về bài toán giải hệ phương trình . . . 4

1.2.1 Khái niệm hệ phương trình . . . 4

1.2.2 Hệ phương trình bậc nhất hai ẩn . . . 4

1.2.3 Hệ phương trình bậc nhất nhiều ẩn . . . 5

1.2.4 Hệ phương trình phi tuyến . . . 5

1.2.5 Một số phương pháp giải hệ phương trình . . . 5

1.3 Vai trò của SymPy trong giải hệ phương trình . . . 6

1.3.1 Hỗ trợ tìm nghiệm của hệ phương trình . . . 6

1.3.2 Hỗ trợ kiểm tra kết quả . . . 6

1.3.3 Hỗ trợ xử lý các hệ phương trình phức tạp . . . 6

1.3.4 Ý nghĩa của việc sử dụng SymPy . . . 6

1.4 Môi trường sử dụng SymPy . . . 7

1.4.1 Cài đặt SymPy . . . 7

1.4.2 Cách viết và chạy chương trình sử dụng SymPy . . . 7

2 Chương 2. CÁC LỆNH SYMPY LIÊN QUAN ĐẾN GIẢI HỆ PHƯƠNG TRÌNH **7**

2.1 Khai báo biến và các đối tượng toán học . . . 7

2.2 Biểu diễn phương trình . . . 8

2.3 Lệnh `solve()` . . . 8

2.4 Lệnh `linsolve()` . . . 9

2.5 Lệnh `nonlinsolve()` . . . 9

2.6 Một số lệnh hỗ trợ . . . 10

2.6.1 Lệnh `sub()` . . . 10

2.6.2 Lệnh `simplify()` . . . 10

2.6.3 Lệnh `checksol()` . . . 10

2.7 Một số lưu ý khi sử dụng SymPy . . . 10

3 Chương 3. Áp dụng Sympy để giải quyết một số bài toán **11**

3.1 Hệ phương trình bậc nhất hai ẩn . . . 11

3.2 Hệ phương trình phi tuyến tính . . . 12

3.3 Hệ phương trình tuyến tính nhiều ẩn . . . 13

3.4 Bài toán thực tế đưa về hệ phương trình . . . 14

4 Kết luận **15**

TÀI LIỆU THAM KHẢO **16**

LỜI NÓI ĐẦU

Trong học tập Toán, việc giải các hệ phương trình là một nội dung quen thuộc và được sử dụng trong nhiều dạng bài khác nhau. Bên cạnh các phương pháp giải truyền thống, hiện nay chúng ta có thể sử dụng các công cụ lập trình để hỗ trợ quá trình tính toán và kiểm tra kết quả. Điều này giúp việc giải toán trở nên thuận tiện hơn, đồng thời tạo cơ hội để vận dụng kiến thức toán học vào công nghệ.

SymPy là một thư viện của Python hỗ trợ tính toán toán học, trong đó có nhiều lệnh phục vụ cho việc giải phương trình và hệ phương trình. Việc tìm hiểu cách sử dụng SymPy không chỉ giúp giải quyết các bài toán nhanh chóng mà còn giúp người học làm quen với việc sử dụng lập trình để hỗ trợ học tập môn Toán.

Từ đó, nhóm lựa chọn đề tài **"Ứng dụng SymPy trong giải hệ phương trình"** với mong muốn tìm hiểu những kiến thức cơ bản về SymPy, các lệnh thường được sử dụng để giải hệ phương trình và cách áp dụng vào một số bài toán cụ thể.

Nội dung đề tài được chia thành ba chương. **Chương 1** giới thiệu khái quát về SymPy và bài toán giải hệ phương trình. **Chương 2** trình bày các lệnh SymPy liên quan đến giải hệ phương trình và một số lưu ý khi sử dụng. **Chương 3** tập trung vào việc áp dụng các lệnh đã tìm hiểu để giải một số hệ phương trình và bài toán cụ thể.

Trong quá trình thực hiện, do kiến thức và kinh nghiệm còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý của giảng viên để đề tài được hoàn thiện hơn.

Nhóm xin chân thành cảm ơn!

1 Chương 1. GIỚI THIỆU CHUNG VỀ SYMPY VÀ GIẢI HỆ PHƯƠNG TRÌNH

1.1 Giới thiệu về SymPy

1.1.1 SymPy là gì?

SymPy là một thư viện mã nguồn mở được sử dụng để thực hiện các phép tính toán học dưới dạng ký hiệu. Khác với việc chỉ tính toán các giá trị số, SymPy cho phép làm việc trực tiếp với các biến, biểu thức và phương trình toán học.
SymPy cung cấp nhiều công cụ phục vụ cho các lĩnh vực khác nhau của toán học như đại số, giải tích, phương trình, hệ phương trình, ma trận,... Trong đó, các chức năng liên quan đến giải phương trình và hệ phương trình là một trong những nội dung được sử dụng trong đề tài này.
Ví dụ, với biểu thức

$$(x+1)^2,$$

SymPy có thể thực hiện khai triển để thu được

$$x^2 + 2x + 1.$$

Ngoài ra, SymPy còn có thể hỗ trợ tìm nghiệm của các phương trình và hệ phương trình thông qua các lệnh phù hợp.

1.1.2 Đặc điểm của SymPy

Một số đặc điểm nổi bật của SymPy gồm:

- **Mã nguồn mở:** SymPy được cung cấp miễn phí và có thể được sử dụng trong nhiều mục đích học tập, nghiên cứu và tính toán.
- **Tính toán ký hiệu:** SymPy có khả năng làm việc với các biến và biểu thức toán học dưới dạng ký hiệu thay vì chỉ xử lý các giá trị số.
- **Hỗ trợ nhiều lĩnh vực toán học:** SymPy cung cấp các công cụ cho đại số, giải tích, phương trình, ma trận, xác suất và một số nội dung toán học khác.
- **Có nhiều lệnh phục vụ giải phương trình:** SymPy cung cấp các lệnh như `solve()`, `linsolve()`, `nonlinsolve()`,... giúp xử lý nhiều dạng phương trình và hệ phương trình.
- **Kết quả có dạng toán học rõ ràng:** Các kết quả được biểu diễn dưới dạng ký hiệu, giúp người sử dụng dễ dàng quan sát và tiếp tục thực hiện các phép tính khác.

1.1.3 Ưu điểm khi sử dụng SymPy trong tính toán toán học

Việc sử dụng SymPy mang lại một số thuận lợi trong quá trình giải quyết các bài toán toán học.

Thứ nhất, SymPy có thể giảm bớt các bước tính toán thủ công. Đối với những biểu thức hoặc hệ phương trình có nhiều bước biến đổi, việc sử dụng các lệnh phù hợp có thể giúp tìm kết quả nhanh chóng hơn.
Thứ hai, SymPy có khả năng thực hiện tính toán chính xác dưới dạng ký hiệu. Điều này đặc biệt hữu ích khi cần làm việc với phân số, căn thức, biểu thức đại số hoặc nghiệm của phương trình.
Thứ ba, SymPy có thể hỗ trợ kiểm tra kết quả. Sau khi giải một bài toán bằng phương pháp truyền thống,

người học có thể sử dụng SymPy để kiểm tra nghiệm và đối chiếu với kết quả đã tìm được.
Cuối cùng, việc sử dụng SymPy giúp kết hợp kiến thức toán học với công cụ lập trình, từ đó tạo thêm một phương pháp tiếp cận đối với các bài toán toán học.
Tuy nhiên, SymPy chủ yếu đóng vai trò là công cụ hỗ trợ tính toán. Người sử dụng vẫn cần hiểu bài toán, xác định đúng các ẩn, lập phương trình và xem xét điều kiện của nghiệm.

1.1.4 Một số lĩnh vực có thể sử dụng SymPy

SymPy được sử dụng trong nhiều nội dung khác nhau của toán học. Một số lĩnh vực tiêu biểu gồm:

- **Đại số:** khai triển, rút gọn và phân tích các biểu thức đại số.
- **Phương trình và hệ phương trình:** tìm nghiệm của phương trình, hệ phương trình tuyến tính và phi tuyến.
- **Giải tích:** tính giới hạn, đạo hàm, nguyên hàm và tích phân.
- **Ma trận:** thực hiện các phép toán với ma trận, tính định thức và một số phép biến đổi liên quan.
- **Đại số tuyến tính:** hỗ trợ các phép tính liên quan đến vector, ma trận và hệ phương trình tuyến tính.

Trong phạm vi đề tài **"Ứng dụng SymPy trong giải hệ phương trình"**, nhóm tập trung chủ yếu vào các chức năng của SymPy liên quan đến biểu diễn và giải hệ phương trình, đặc biệt là các lệnh được sử dụng trong Chương 2 và các bài toán áp dụng ở Chương 3.

1.2 Giới thiệu về bài toán giải hệ phương trình

1.2.1 Khái niệm hệ phương trình

Hệ phương trình là tập hợp gồm từ hai phương trình trở lên có chung các ẩn. Nghiệm của hệ phương trình là một bộ giá trị của các ẩn đồng thời thỏa mãn tất cả các phương trình trong hệ.
Ví dụ, xét hệ phương trình:

$$\begin{cases} x + y = 5 \\ 2x - y = 1. \end{cases}$$

Với $x = 2, y = 3$, ta có:

$$2 + 3 = 5$$

và

$$2.2 - 3 = 1.$$

Do đó, $(x, y) = (2, 3)$ là nghiệm của hệ phương trình.

1.2.2 Hệ phương trình bậc nhất hai ẩn

Hệ phương trình bậc nhất hai ẩn có dạng tổng quát:

$$\begin{cases} a_1 x + b_1 y = c_1, \\ a_2 x + b_2 y = c_2, \end{cases}$$

trong đó x, y là các ẩn và $a_1, b_1, a_2, b_2, c_1, c_2$ là các số đã biết.
Tùy thuộc vào mối quan hệ giữa các hệ số, hệ phương trình có thể có một nghiệm, vô nghiệm hoặc có vô số

nghiệm.
Ví dụ:

$$\begin{cases} x + y = 5, \\ 2x - y = 1. \end{cases}$$

Hệ phương trình trên có nghiệm duy nhất:

$$(x, y) = (2, 3).$$

Đây là dạng hệ phương trình cơ bản và thường được sử dụng để minh họa cách sử dụng các lệnh giải hệ phương trình trong SymPy.

1.2.3 Hệ phương trình bậc nhất nhiều ẩn

Hệ phương trình bậc nhất nhiều ẩn là hệ trong đó các phương trình đều có bậc nhất đối với các ẩn. Với ba ẩn x, y, z, hệ có thể có dạng:

$$\begin{cases} a_1x + b_1y + c_1z = d_1, \\ a_2x + b_2y + c_2z = d_2, \\ a_3x + b_3y + c_3z = d_3. \end{cases}$$

Ví dụ:

$$\begin{cases} x + y + z = 6, \\ 2x - y + z = 3, \\ x + 2y - z = 2. \end{cases}$$

So với hệ hai ẩn, hệ nhiều ẩn thường có nhiều bước tính toán hơn khi giải bằng phương pháp thủ công. Vì vậy, các công cụ hỗ trợ tính toán như SymPy có thể giúp quá trình tìm nghiệm trở nên thuận tiện hơn.

1.2.4 Hệ phương trình phi tuyến

Hệ phương trình phi tuyến là hệ trong đó có ít nhất một phương trình không phải là phương trình bậc nhất đối với các ẩn. Hệ có thể xuất hiện các biểu thức như x^2, y^2, xy, $\sqrt{x}$,...
Ví dụ:

$$\begin{cases} x^2 + y^2 = 25, \\ x + y = 7. \end{cases}$$

Đây là một hệ phương trình phi tuyến vì phương trình thứ nhất chứa x^2 và y^2.
Việc giải hệ phương trình phi tuyến thường phức tạp hơn so với hệ phương trình bậc nhất. Trong trường hợp này, SymPy có thể hỗ trợ tìm các nghiệm của hệ thông qua những lệnh phù hợp.

1.2.5 Một số phương pháp giải hệ phương trình

Trong toán học, hệ phương trình có thể được giải bằng nhiều phương pháp khác nhau. Đối với hệ phương trình bậc nhất, một số phương pháp thường gặp là phương pháp thế và phương pháp cộng đại số. Ngoài ra, có thể sử dụng các công cụ tính toán để hỗ trợ tìm và kiểm tra nghiệm.
Trong đề tài này, nhóm tập trung vào việc sử dụng SymPy để hỗ trợ giải các hệ phương trình. Các lệnh và cách sử dụng cụ thể sẽ được trình bày ở Chương 2, sau đó được vận dụng vào các bài toán cụ thể ở Chương 3.

1.3 Vai trò của SymPy trong giải hệ phương trình

Việc giải hệ phương trình bằng các phương pháp truyền thống thường đòi hỏi người giải thực hiện nhiều bước biến đổi và tính toán. Khi số lượng ẩn hoặc độ phức tạp của hệ tăng lên, quá trình này có thể mất nhiều thời gian và dễ xảy ra sai sót. Trong trường hợp đó, SymPy có thể được sử dụng như một công cụ hỗ trợ tính toán.
Ví dụ, với hệ:

$$\begin{cases} x + y = 5 \\ 2x - y = 1, \end{cases}$$

có thể biểu diễn các phương trình trong SymPy để chuẩn bị cho quá trình giải.

1.3.1 Hỗ trợ tìm nghiệm của hệ phương trình

SymPy cung cấp các lệnh phù hợp để tìm nghiệm của nhiều dạng hệ phương trình khác nhau. Đối với hệ phương trình tuyến tính, có thể sử dụng các lệnh như `solve()` hoặc `linsolve()`. Đối với một số hệ phương trình phi tuyến, có thể sử dụng `solve()` hoặc `nonlinsolve()` tùy trường hợp. Ví dụ, với hệ:

$$\begin{cases} x + y = 5, \\ 2x - y = 1, \end{cases}$$

SymPy có thể tìm được nghiệm:

$$x = 2, \quad y = 3.$$

Các lệnh cụ thể và cách sử dụng sẽ được trình bày ở Chương 2.

1.3.2 Hỗ trợ kiểm tra kết quả

Bên cạnh việc tìm nghiệm, SymPy còn có thể hỗ trợ người sử dụng kiểm tra lại kết quả. Sau khi tìm được nghiệm, có thể thay các giá trị đó vào các phương trình ban đầu để kiểm tra xem chúng có thỏa mãn hệ hay không.
Điều này giúp người học có thể đối chiếu kết quả khi giải bằng phương pháp truyền thống và phát hiện những sai sót trong quá trình tính toán nếu có.

1.3.3 Hỗ trợ xử lý các hệ phương trình phức tạp

Đối với những hệ phương trình có nhiều ẩn hoặc có dạng phi tuyến, việc giải bằng phương pháp thủ công có thể cần nhiều bước biến đổi. SymPy giúp thực hiện phần tính toán một cách nhanh chóng và cung cấp kết quả dưới dạng phù hợp để người sử dụng tiếp tục phân tích.
Tuy nhiên, SymPy chỉ đóng vai trò hỗ trợ quá trình giải toán. Người sử dụng vẫn cần tự phân tích đề bài, xác định các ẩn, thiết lập hệ phương trình và xem xét điều kiện của nghiệm. Vì vậy, việc sử dụng SymPy cần được kết hợp với kiến thức toán học để đảm bảo kết quả thu được phù hợp với bài toán.

1.3.4 Ý nghĩa của việc sử dụng SymPy

Việc sử dụng SymPy trong giải hệ phương trình giúp kết hợp kiến thức toán học với công cụ lập trình, đồng thời tạo điều kiện để người học làm quen với cách sử dụng máy tính trong quá trình giải quyết bài toán. Qua đó, SymPy có thể được xem như một công cụ hỗ trợ tính toán, giúp giảm bớt các thao tác thủ công và thuận tiện hơn trong việc kiểm tra kết quả.

1.4 Môi trường sử dụng SymPy

Để sử dụng SymPy giải các bài toán về hệ phương trình, cần chuẩn bị môi trường lập trình và cài đặt thư viện SymPy. Trong đề tài này, nhóm sử dụng Python làm môi trường để chạy các chương trình có sử dụng SymPy.

1.4.1 Cài đặt SymPy

SymPy có thể được cài đặt thông qua công cụ `pip`. Trong cửa sổ Command Prompt (CMD), sử dụng lệnh `pip install sympy`.
Sau khi quá trình cài đặt hoàn tất, thư viện SymPy có thể được sử dụng trong các chương trình Python.

1.4.2 Cách viết và chạy chương trình sử dụng SymPy

Có thể viết chương trình SymPy bằng các trình soạn thảo như Notepad, IDLE hoặc PyCharm và lưu chương trình với phần mở rộng `.py`.
Ví dụ, tạo một tệp có tên `test.py` với nội dung:

```
import sympy as sp

x = sp.symbols('x')
pt = sp.Eq(x + 2, 5)
kq = sp.solve(pt,x)
print(kq)
```

Sau khi lưu tệp, có thể chạy chương trình bằng CMD với lệnh `python test.py`
Kết quả nhận được: `[3]`
Qua ví dụ trên, có thể thấy quy trình sử dụng SymPy cơ bản gồm: cài đặt thư viện → viết chương trình → chạy chương trình → quan sát kết quả. Trong các phần tiếp theo, nhóm sẽ sử dụng quy trình này để thực hiện các lệnh liên quan đến giải hệ phương trình.

2 Chương 2. CÁC LỆNH SYMPY LIÊN QUAN ĐẾN GIẢI HỆ PHƯƠNG TRÌNH

2.1 Khai báo biến và các đối tượng toán học

Trước khi thực hiện giải phương trình hay hệ phương trình trong SymPy, cần khai báo các biến toán học dưới dạng đối tượng ký hiệu (symbolic variables). Việc khai báo này giúp SymPy nhận biết các ẩn và thực hiện các phép tính đại số trên chúng.
Hàm `symbols()`
Công dụng: Dùng để khai báo một hoặc nhiều biến ký hiệu cùng lúc.
Cú pháp: `sp.symbols('x y z')`
Ví dụ:

```
import sympy as sp
x, y, z = sp.symbols('x y z')
```

Sau khi khai báo, các biến `x, y, z` có thể được sử dụng trong việc xây dựng biểu thức, phương trình và hệ phương trình.
Lưu ý: Cần khai báo các biến trước khi sử dụng chúng trong các phép tính của SymPy.

2.2 Biểu diễn phương trình

Trong SymPy, phương trình có thể được biểu diễn bằng hai cách phổ biến:

Cách 1: Sử dụng hàm `Eq()`

Công dụng: Dùng để biểu diễn một phương trình có dạng hai vế bằng nhau.

Cú pháp: `sp.Eq(expr1, expr2)`

Trong đó, `expr1` và `expr2` lần lượt là biểu thức ở vế trái và vế phải của phương trình.

Ví dụ: Biểu diễn phương trình $x + y = 5$ bằng SymPy theo cú pháp sau:

```
pt = sp.Eq(x + y, 5
```

Cách 2: Biểu diễn phương trình dưới dạng biểu thức bằng 0

Ta chuyển tất cả các hạng tử sang vế trái để đưa phương trình về dạng $f(x, y) = 0$.

Ví dụ, phương trình $x + y = 5$ được biểu diễn bằng lệnh:

```
pt = x + y - 5
```

Cả hai cách biểu diễn đều có thể được sử dụng trong các lệnh giải phương trình và hệ phương trình của SymPy.

2.3 Lệnh `solve()`

`solve()` là lệnh tổng quát được sử dụng để giải phương trình và hệ phương trình. Lệnh này có thể áp dụng cho nhiều dạng phương trình khác nhau, bao gồm cả phương trình tuyến tính và phi tuyến.

Cú pháp: `sp.solve(equations, variables)`

Trong đó:

`equations`: Phương trình hoặc danh sách các phương trình cần giải.

`variables`: Biến hoặc danh sách các biến cần tìm nghiệm.

Ví dụ:

Giải hệ phương trình:

$$\begin{cases} x + y = 5, \\ 2x - y = 1. \end{cases}$$

Chương trình:

```
import sympy as sp
x, y = sp.symbols('x y')
pt1 = sp.Eq(x + y, 5)
pt2 = sp.Eq(2*x - y, 1)
kq = sp.solve([pt1, pt2], (x, y), dict=True)
print(kq)
```

Kết quả: `[x: 2, y: 3]`

Như vậy, hệ phương trình có nghiệm duy nhất:

$$(x, y) = (2, 3).$$

Ở ví dụ trên, tùy chọn `dict=True` giúp kết quả được trả về dưới dạng danh sách các từ điển, trong đó mỗi biến được gắn với giá trị nghiệm tương ứng.

Lưu ý: Nếu không sử dụng `dict=True`, kết quả trả về của `solve()` có thể có dạng khác nhau tùy thuộc vào bài toán.

2.4 Lệnh `linsolve()`

`linsolve()` là lệnh chuyên dụng để giải các hệ phương trình tuyến tính. Lệnh này hỗ trợ giải hệ phương trình có nhiều ẩn và có thể biểu diễn hệ thông qua danh sách phương trình hoặc ma trận hệ số.

Cú pháp: `sp.linsolve([eq1, eq2, ...], (x, y, ...))`

Ví dụ minh họa:

Ví dụ:

Giải hệ phương trình:

$$\begin{cases} x + y + z = 6, \\ 2x - y + z = 3, \\ x + 2y - z = 2. \end{cases}$$

Chương trình:

```
import sympy as sp
x, y, z = sp.symbols('x y z')
pt1 = x + y + z - 6
pt2 = 2*x - y + z - 3
pt3 = x + 2*y - z - 2
kq = sp.linsolve([pt1, pt2, pt3], (x, y, z))
print(kq)
```

Kết quả: `(1, 2, 3)`

Vậy hệ phương trình có nghiệm:

$$(x, y, z) = (1, 2, 3).$$

Cú pháp dạng ma trận: Đối với hệ phương trình tuyến tính có dạng:

$$AX = B,$$

trong đó A là ma trận hệ số, X là vector các ẩn và B là vector các hệ số tự do, có thể sử dụng:

`sp.linsolve((A, B), (x, y, ...))`

Cách biểu diễn này thuận tiện khi giải các hệ phương trình tuyến tính có nhiều ẩn. **Lưu ý:** Kết quả của `linsolve()` được trả về dưới dạng `FiniteSet`, trong đó mỗi phần tử là một bộ nghiệm tương ứng với các ẩn đã khai báo.

2.5 Lệnh `nonlinsolve()`

`nonlinsolve()` được sử dụng để giải các hệ phương trình phi tuyến, tức là những hệ có ít nhất một phương trình không phải phương trình bậc nhất đối với các ẩn.

Lệnh này có thể xử lý các hệ chứa lũy thừa, tích giữa các biến và một số dạng biểu thức phi tuyến khác.

Cú pháp: `nonlinsolve([eq1, eq2, ...], (x, y, ...))`

Trong đó:

`eq1, eq2, ...`: Các phương trình của hệ.

`x, y...`: Các biến cần tìm nghiệm.

Ví dụ: Giải hệ phương trình $\begin{cases} \mathrm{x}^2 + y^2 = 25 \\ x + y = 7. \end{cases}$

Chương trình:

```
import sympy as sp
x, y = sp.symbols('x y')
pt1 = x**2 + y**2 - 25
pt2 = x + y - 7
kq = sp.nonlinsolve([pt1, pt2], (x, y))
print(kq)
```
Kết quả: `(3, 4), (4, 3)`

2.6 Một số lệnh hỗ trợ

Ngoài các lệnh trực tiếp phục vụ việc giải hệ phương trình, SymPy còn cung cấp một số lệnh hỗ trợ quá trình tính toán và kiểm tra kết quả.

2.6.1 Lệnh `sub()`

Công dụng: Dùng để thay thế giá trị của một biến trong biểu thức bằng một giá trị hoặc biểu thức khác.
Cú pháp: `expr.subs(variable, value)`
Ví dụ:
```
expr = x + y
kq = expr.subs(x: 2, y: 3)
print(kq)
```

2.6.2 Lệnh `simplify()`

Công dụng: Dùng để rút gọn biểu thức toán học.
Cú pháp: `simplify(expr)`
Ví dụ:
```
expr = (x**2 - 1)/(x - 1)
kq = sp.simplify(expr)
print(kq)
```

2.6.3 Lệnh `checksol()`

Công dụng: Dùng để kiểm tra một giá trị hoặc một bộ giá trị có thỏa mãn phương trình hay không.
Cú pháp: `checksol(equation, solution)`
Ví dụ:
```
pt = x + y - 5
kq = sp.checksol(pt, x: 2, y: 3)
print(kq)
```
Kết quả: `True` cho biết bộ giá trị đã cho thỏa mãn phương trình đang xét.

2.7 Một số lưu ý khi sử dụng SymPy

Trong quá trình sử dụng SymPy để giải hệ phương trình, cần chú ý một số vấn đề sau:
Khai báo biến: Cần khai báo các biến bằng `sp.symbols()` trước khi sử dụng.
Biểu diễn phương trình: Có thể sử dụng `sp.Eq()` để biểu diễn phương trình hoặc đưa phương trình về dạng biểu thức bằng 0.
Phép toán: Sử dụng `**` cho phép lũy thừa, `*` cho phép nhân và `/` cho phép chia.

Chọn lệnh phù hợp: Sử dụng `sp.linsolve()` cho hệ tuyến tính, `sp.nonlinsolve()` cho hệ phi tuyến và `sp.solve()` cho nhiều dạng phương trình khác nhau.
Kiểm tra nghiệm: Sau khi tìm được nghiệm, nên thay nghiệm vào hệ phương trình ban đầu để kiểm tra kết quả.
Nghiệm của hệ: Hệ phương trình có thể có một nghiệm, vô nghiệm hoặc vô số nghiệm. Cần dựa vào kết quả của SymPy để đưa ra kết luận.

3 Chương 3. Áp dụng Sympy để giải quyết một số bài toán

3.1 Hệ phương trình bậc nhất hai ẩn

Bài 1: Giải hệ phương trình:

$$\begin{cases} -x + 3y = -10, \\ x - 5y = 16. \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> pt1 = sp.Eq(-x+3*y,-10)
>>> pt2 = sp.Eq(x-5*y,16)
>>> kq = sp.solve((pt1,pt2),(x,y))
>>> print(kq)
{x: 1, y: -3}
```

Bài 2: Giải hệ phương trình:

$$\begin{cases} \frac{1}{x} - \frac{1}{y} = 1 \\ \frac{2}{x} + \frac{4}{y} = 5. \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> pt1 = sp.Eq((1/x)-(1/y),1)
>>> pt2 = sp.Eq((2/x)+(4/y),5)
>>> kq = sp.solve((pt1,pt2),(x,y))
>>> print(kq)
[(2/3, 2)]
```

Bài 3: Giải hệ phương trình:

$$\begin{cases} \dfrac{2}{x+1}+\dfrac{3}{y}=1 \\ \dfrac{2}{x+1}+\dfrac{5}{y}=1 \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> pt1 = sp.Eq((2/(x+1))+((3/y)),1)
>>> pt2 = sp.Eq((2/(x+1))+(5/y),1)
>>> kq = sp.solve((pt1,pt2),(x,y))
>>> print(kq)
[]
```

3.2 Hệ phương trình phi tuyến tính

Bài 1: Giải hệ phương trình:

$$\begin{cases} x^2+y=5 \\ x+y^2=7 \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> pt1 = sp.Eq(x**2 + y,5)
>>> pt2 = sp.Eq(x+y**2,7)
>>> kq = sp.solve((pt1,pt2),(x,y))
>>> print(kq)
[(7 - (-sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/2 - sqrt(56/3 - 2*(38891/432 + sqrt
(611535)*I/144)**(1/3) + 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/
432 + sqrt(611535)*I/144)**(1/3)))/2)**2, -sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/
2 - sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) + 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(6115
35)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2), (7 - (sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(388
91/432 + sqrt(611535)*I/144)**(1/3))/2 - sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) - 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/14
4)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2)**2, sqrt(28/3 + 362/(9*(38891/432 + sq
rt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/2 - sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) - 2/sqrt(28/3 + 362
/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2), (7
 - (sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) - 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(61153
5)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2 + sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432
 + sqrt(611535)*I/144)**(1/3))/2)**2, sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) - 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**
(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2 + sqrt(28/3 + 362/(9*(38891/432 + sqrt(6115
35)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/2), (7 - (sqrt(56/3 - 2*(38891/432 + sqrt(611535)*I/144)**(1/3) + 2/sqrt(28/3 + 362/
(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)))/2 - sqrt
(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/2)**2, sqrt(56/3 - 2*(38891/432 + sqrt(611535)*
I/144)**(1/3) + 2/sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3)) - 362/(9*(38891/432 + sqr
t(611535)*I/144)**(1/3)))/2 - sqrt(28/3 + 362/(9*(38891/432 + sqrt(611535)*I/144)**(1/3)) + 2*(38891/432 + sqrt(611535)*I/144)**(1/3))/2)]
>>>
```

Khi giải các hệ phương trình phi tuyến tính bằng các câu lệnh giải đại số thông thường (như solve), kết quả thu được thường ở dạng nghiệm chính xác (biểu thức căn thức, phân thức phức tạp) khiến lời giải trở nên rất dài và khó quan sát. Do đó, việc chuyển sang sử dụng lệnh nsolve là giải pháp tối ưu nhằm tìm nghiệm số xấp xỉ, giúp kết quả hiển thị gọn gàng, rõ ràng và thuận tiện hơn cho việc tính toán tiếp theo.

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> pt1 = sp.Eq(x**2 + y,5)
>>> pt2 = sp.Eq(x+y**2,7)
>>> kq = sp.nsolve((pt1,pt2),(x,y),(1,2))
>>> print(kq)
Matrix([[1.63844330017618], [2.31550355210780]])
```

Bài 2: Giải hệ phương trình:

$$\begin{cases} x^2 + y^2 = 2x^2y^2 \\ (x+y)(1+xy) = 4x^2y^2 \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x,y = sp.symbols('x y')
>>> eq1 = x**2 + y**2 - 2*x**2*y**2
>>> eq2 = (x + y)*(1 + x*y) - 4*x**2*y**2
>>> kq = sp.nsolve((eq1, eq2), (x, y), (1, 1))
>>> print(kq)
Matrix([[1.00000000000000], [1.00000000000000]])
```

3.3 Hệ phương trình tuyến tính nhiều ẩn

Bài 1: Giải hệ phương trình:

$$\begin{cases} 3x + 2y - 5z = -3 \\ 2x + y + 3z = 11 \\ 4x + 3y - 12z = -15 \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x, y, z = sp.symbols('x y z')
>>> pt1 = 3*x + 2*y - 5*z + 3
>>> pt2 = 2*x + y + 3*z - 11
>>> pt3 = 4*x + 3*y - 12*z + 15
>>> kq = sp.solve((pt1, pt2, pt3), (x, y, z))
>>> print(kq)
{x: 3, y: -1, z: 2}
```

Bài 2: Giải hệ phương trình:

$$\begin{cases} 2x_1 - x_2 + 4x_3 + 2x_4 = 2 \\ 3x_1 - 2x_2 + 7x_3 + 2x_4 = 1 \\ 5x_1 - 3x_2 + 7x_3 + 6x_4 = 5 \end{cases}$$

Giải

```
>>> import sympy as sp
>>> x1, x2, x3, x4 = sp.symbols('x1 x2 x3 x4')
>>> pt1 = 2*x1 - x2 + 4*x3 + 2*x4 - 2
>>> pt2 = 3*x1 - 2*x2 + 7*x3 + 2*x4 - 1
>>> pt3 = 5*x1 - 3*x2 + 7*x3 + 6*x4 - 5
>>> kq = sp.solve((pt1, pt2, pt3), (x1, x2, x3, x4))
>>> print(kq)
{x1: 7/2 - 5*x4/2, x2: 3 - x4, x3: x4/2 - 1/2}
```

3.4 Bài toán thực tế đưa về hệ phương trình

Bài 1: Một trường THCS mua 500 quyển vở để làm phần thưởng cho học sinh. Giá bán của mỗi quyển vở loại I, loại II lần lượt là 8000 và 9000 đồng. Hỏi trường đã mua mỗi loại bao nhiêu quyển vở, biết rằng tổng số tiền đã dùng là $4.200.000$ đồng.

Hướng dẫn giải:

Bước 1: Đặt ẩn

Gọi x, y lần lượt là số quyển vở loại I, loại II đã mua (điều kiện: $x, y \in \mathbb{N}^*$).

Bước 2: Lập hệ phương trình

Tổng số vở đã mua là 500 quyển nên ta có phương trình:

$$x + y = 500 \tag{1}$$

Tổng số tiền đã dùng để mua vở là $4.200.000$ đồng (tương đương 4200 nghìn đồng) nên ta có phương trình:

$$8000x + 9000y = 4200000 \iff 8x + 9y = 4200 \tag{2}$$

Từ (1) và (2) ta có hệ phương trình:

$$\begin{cases} x + y = 500 \\ 8x + 9y = 4200 \end{cases} \tag{3}$$

Bước 3: Giải hệ phương trình

```
>>> import sympy as sp
>>> x, y = sp.symbols('x y')
>>> eq1 = x + y - 500
>>> eq2 = 8*x + 9*y - 4200
>>> kq = sp.solve((eq1, eq2), (x, y))
>>> print(kq)
{x: 300, y: 200}
```

Bước 4: Kết luận

Vậy số quyển vở loại I và loại II mà trường đã mua lần lượt là 300 quyển và 200 quyển.

Bài 2: Bác Phương chia số tiền 800 triệu đồng của mình cho hai khoản đầu tư. Sau một năm, tổng số tiền lãi thu được là 54 triệu đồng. Lãi suất cho khoản đầu tư thứ nhất là 6%/năm và khoản đầu tư thứ hai là 8%/năm. Tính số tiền bác Phương đầu tư cho mỗi khoản.

Hướng dẫn giải:

Bước 1: Đặt ẩn và điều kiện

Gọi x (triệu đồng) là số tiền bác Phương đầu tư cho khoản thứ nhất, điều kiện: $0 < x < 800$.

Gọi y (triệu đồng) là số tiền bác Phương đầu tư cho khoản thứ hai.

Bước 2: Lập hệ phương trình

Vì tổng số tiền đầu tư là 800 triệu đồng nên ta có phương trình:

$$x + y = 800 \quad (1)$$

Số tiền lãi từ khoản thứ nhất là $0{,}06x$ (triệu đồng), số tiền lãi từ khoản thứ hai là $0{,}08y$ (triệu đồng). Tổng số tiền lãi là 54 triệu đồng nên ta có phương trình:

$$0{,}06x + 0{,}08y = 54 \quad (2)$$

Từ (1) và (2) ta có hệ phương trình:

$$\begin{cases} x + y = 800 \\ 0{,}06x + 0{,}08y = 54 \end{cases}$$

Bước 3: Giải hệ phương trình

```
{x: 300, y: 200}
>>> import sympy as sp
>>> x, y = sp.symbols('x y')
>>> eq1 = x + y - 800
>>> eq2 = 0.06*x + 0.08*y - 54
>>> kq = sp.solve((eq1, eq2), (x, y))
>>> print(kq)
{x: 500.000000000000, y: 300.000000000000}
```

Bước 4: Kết luận

Vậy số tiền bác Phương đầu tư cho khoản thứ nhất là 500 triệu đồng và khoản thứ hai là 300 triệu đồng.

4 Kết luận

Qua quá trình tìm hiểu và thực hiện đề tài **"Ứng dụng SymPy trong giải hệ phương trình"**, nhóm đã có cơ hội nghiên cứu những kiến thức cơ bản về thư viện SymPy và cách vận dụng thư viện này để giải quyết các bài toán liên quan đến hệ phương trình. Đề tài đã trình bày khái quát về SymPy, giới thiệu các lệnh thường dùng trong giải hệ phương trình như `solve()`, `linsolve()` và `nonlinsolve()`, đồng thời áp dụng các lệnh đó vào nhiều dạng bài toán khác nhau như hệ phương trình bậc nhất hai ẩn, hệ phương trình tuyến tính nhiều ẩn, hệ phương trình phi tuyến và một số bài toán thực tế.

Thông qua các ví dụ minh họa, có thể thấy SymPy là một công cụ hỗ trợ tính toán hiệu quả, giúp rút ngắn thời gian giải toán, hạn chế sai sót trong quá trình tính toán và hỗ trợ kiểm tra kết quả một cách nhanh chóng. Bên cạnh đó, việc sử dụng SymPy còn góp phần giúp người học làm quen với lập trình Python và nâng cao khả năng kết hợp giữa kiến thức toán học với công nghệ trong học tập.

Tuy nhiên, SymPy chỉ đóng vai trò là công cụ hỗ trợ. Để sử dụng hiệu quả, người học vẫn cần nắm vững kiến thức toán học, biết cách phân tích đề bài, xây dựng mô hình toán học và đánh giá tính hợp lý của kết quả thu được. Vì vậy, việc kết hợp giữa tư duy toán học và công cụ lập trình sẽ mang lại hiệu quả cao hơn trong quá trình giải quyết các bài toán.

TÀI LIỆU THAM KHẢO

1. Hà Huy Khoái (Tổng chủ biên). *Toán 9, Tập một: Kết nối tri thức với cuộc sống*. Nhà xuất bản Giáo dục Việt Nam, 2024.
2. Hà Huy Khoái (Tổng chủ biên). *Toán 10 – Kết nối tri thức với cuộc sống*. Nhà xuất bản Giáo dục Việt Nam, 2022.
3. Nguyễn Duy Thuận (chủ biên), Phi Mạnh Ban, Nông Quốc Chinh, *Đại số tuyến tính*, Nhà xuất bản Đại học Sư phạm.
4. Trần Đình Quang, *Lập trình Python từ cơ bản đến nâng cao và ứng dụng tính toán khoa học*. Nhà xuất bản Thông tin và Truyền thông, 2021.
5. SymPy Development Team (không ghi năm), Solvers [Các công cụ giải phương trình]. Truy cập tại: https://docs.sympy.org/latest/modules/solvers/solvers.html